

OF ACTIVATION FUNCTIONS FOR IMAGE CLASSIFICATION

Implementation Report Based on the Uploaded Jupyter Notebook and Research Paper

Dataset: CIFAR-10 | Framework: TensorFlow/Keras | Environment: Jupyter Notebook on Apple M3 GPU

1. Executive Summary

This report documents the implementation and experimental evaluation of a ResNet-18 image-classification model using seven activation functions: ReLU, ELU, GELU, Swish, Tanh, Softsign, and SoftPlus. The implementation follows the experimental direction of the supplied research paper, which evaluates ResNet-18 on CIFAR-10 with different activation functions. The submitted notebook normalizes CIFAR-10 images, creates a 45,000/5,000 training/validation split, trains separate models, evaluates them on the 10,000-image test set, compares the results with the paper, and performs detailed evaluation of the best model.

In the final 15-epoch experiment, Swish produced the highest test accuracy in the submitted implementation at 69.99%. The paper reports 70.63% for Swish, so the difference is -0.64 percentage points. The paper's overall best result is ELU at 72.86%, whereas Swish is the best activation in the submitted 15-epoch run.

2. Research Paper Basis

The supplied paper is titled “ResNet-18 comparative analysis of various activation functions for image classification” by Gaurav Kumar Pandey and Sumit Srivastava. Its stated objective is to evaluate ResNet-18 with different activation functions on an image-classification task and compare predictive performance and training behavior.

The paper evaluates seven activation functions: Tanh, Softsign, ReLU, SoftPlus, ELU, GELU, and Swish. It describes CIFAR-10 as a 60,000-image dataset of 32×32 color images, with 50,000 training images and 10,000 test images. It also describes an SGD optimizer with learning rate 0.1, momentum 0.9, and decay 5×10^{-4} .

The paper's reported accuracy table is reproduced below for comparison.

Activation	Paper Accuracy (%)
Tanh	68.72
Softsign	66.89
ReLU	69.55
SoftPlus	65.79
ELU	72.86
GELU	71.18
Swish	70.63

3. Objectives

- Implement a ResNet-18 model for CIFAR-10 classification.
- Evaluate seven activation functions under a common experimental setup.
- Measure test accuracy and training time.
- Compare the submitted results with the research-paper results.
- Identify the best activation function in the implemented experiment.
- Perform detailed evaluation of the selected model using predictions, classification metrics, and a confusion matrix.
- Save the final best model for future use.

4. Experimental Environment

Item	Recorded configuration
Framework	TensorFlow / Keras
TensorFlow version in final environment	2.18.1
NumPy	2.0.2
Pandas	3.0.5
Hardware acceleration	GPU available; Apple Metal GPU reported by TensorFlow
Input image size	$32 \times 32 \times 3$
Number of classes	10
Final epochs	15
Batch size	128
Learning rate	0.1
Momentum	0.9
Decay	0.0005
Loss	Categorical Crossentropy
Optimizer	SGD with inverse-time learning-rate schedule

5. Dataset and Preprocessing

The notebook loads CIFAR-10 and verifies 50,000 training images and 10,000 test images. The training set is then split into 45,000 training images and 5,000 validation images. Pixel values are converted to float32 and scaled from 0–255 to the range 0–1. Labels are one-hot encoded into ten classes.

```

# Dataset normalization
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Validation split
x_val = x_train[-5000:]
y_val = y_train[-5000:]
x_train = x_train[:-5000]
y_train = y_train[:-5000]

# One-hot encoding
y_train = to_categorical(y_train, num_classes=10)
y_val = to_categorical(y_val, num_classes=10)
y_test = to_categorical(y_test, num_classes=10)

OUTPUT
x_train: (45000, 32, 32, 3)
y_train: (45000,)
x_val: (5000, 32, 32, 3)
y_val: (5000,)
x_test: (10000, 32, 32, 3)
y_test: (10000,)

Pixel range: 0.0 to 1.0
Example label: 6
Example class: frog

Training labels: (45000, 10)
Validation labels: (5000, 10)
Test labels: (10000, 10)

```

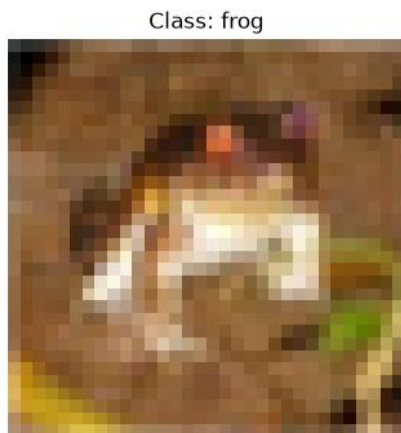


Figure 1. Example CIFAR-10 training image displayed by the notebook (frog class).

6. ResNet-18 Architecture Implemented

The notebook implements a residual CNN with an initial 7×7 convolution (64 filters, stride 2), max pooling, four residual groups with two residual blocks per group, channel sizes 64/128/256/512, global average pooling, and a 10-unit softmax output layer. Each residual block uses two 3×3 convolutions with batch normalization and a projection shortcut when the spatial size or channel count changes.

```

def residual_block(x, filters, stride=1, activation="relu"):
    shortcut = x

    x = Conv2D(filters, 3, strides=stride,
                padding="same", use_bias=False)(x)
    x = BatchNormalization()(x)
    x = apply_activation(x, activation)

    x = Conv2D(filters, 3, strides=1,
                padding="same", use_bias=False)(x)
    x = BatchNormalization()(x)

    if stride != 1 or shortcut.shape[-1] != filters:
        shortcut = Conv2D(filters, 1, strides=stride,
                           padding="same", use_bias=False)(shortcut)
        shortcut = BatchNormalization()(shortcut)

    x = Add()([x, shortcut])
    x = apply_activation(x, activation)
    return x

# Initial stem
x = Conv2D(64, 7, strides=2, padding="same",
           use_bias=False)(inputs)
x = BatchNormalization()(x)
x = apply_activation(x, activation)

x = MaxPooling2D(pool_size=3, strides=2,
                  padding="same")(x)

# Residual groups
# 64 -> 128 -> 256 -> 512 filters
# two residual blocks in each group

x = GlobalAveragePooling2D()(x)
outputs = Dense(10, activation="softmax")(x)

```

The notebook's recorded model summary reports 11,191,242 total parameters, including 11,181,642 trainable and 9,600 non-trainable parameters.

```

OUTPUT
Total params: 11,191,242 (42.69 MB)
Trainable params: 11,181,642 (42.65 MB)
Non-trainable params: 9,600 (37.50 KB)

```

7. Activation Functions

The same model-building function is parameterized by activation name. The notebook supports ReLU, ELU, GELU, Swish, Tanh, Softsign, and SoftPlus, matching the seven activation functions investigated in the supplied paper.

```
def apply_activation(x, activation):
    if activation == "relu":
        return tf.keras.layers.ReLU()(x)
    elif activation == "elu":
        return tf.keras.layers.ELU()(x)
    elif activation == "gelu":
        return tf.keras.layers.Activation(
            tf.keras.activations.gelu)(x)
    elif activation == "swish":
        return tf.keras.layers.Activation(
            tf.keras.activations.swish)(x)
    elif activation == "tanh":
        return tf.keras.layers.Activation("tanh")(x)
    elif activation == "softsign":
        return tf.keras.layers.Activation("softsign")(x)
    elif activation == "softplus":
        return tf.keras.layers.Activation("softplus")(x)
    else:
        raise ValueError(f"Unknown activation: {activation}")
```

8. Training Configuration

```
EPOCHS = 15
BATCH_SIZE = 128

LEARNING_RATE = 0.1
MOMENTUM = 0.9
DECAY = 5e-4

lr_schedule = tf.keras.optimizers.schedules.InverseTimeDecay(
    initial_learning_rate=LEARNING_RATE,
    decay_steps=1,
    decay_rate=DECAY,
    staircase=False
)

optimizer = tf.keras.optimizers.SGD(
    learning_rate=lr_schedule,
    momentum=MOMENTUM
)

model.compile(
    optimizer=optimizer,
    loss=tf.keras.losses.CategoricalCrossentropy(),
    metrics=["accuracy"]
)

OUTPUT
FINAL CONFIGURATION
Epochs: 15
Batch size: 128
Learning rate: 0.1
Momentum: 0.9
Decay: 0.0005

Optimizer function created successfully.
Model compiled successfully.
```

9. Experimental Procedure

1. Build a fresh ResNet-18 model for each activation function.
2. Create a fresh SGD optimizer for each model.

3. Train on 45,000 training images and monitor 5,000 validation images.
4. Use 15 epochs and batch size 128 for the final comparative run.
5. Evaluate each trained model on the fixed 10,000-image test set.
6. Record test accuracy and training time.
7. Rank the seven models by test accuracy.
8. Select the highest-accuracy model for detailed evaluation.

10. Final Comparative Results

Rank	Activation	Our Accuracy (%)	Paper (%)	Difference (percentage points)
1	Swish	69.99	70.63	-0.64
2	ELU	69.89	72.86	-2.97
3	GELU	68.73	71.18	-2.45
4	ReLU	66.41	69.55	-3.14
5	SoftPlus	66.36	65.79	+0.57
6	Softsign	65.41	66.89	-1.48
7	Tanh	63.74	68.72	-4.98

The notebook therefore identifies Swish as the best activation in the final 15-epoch experiment, with 69.99% test accuracy. The research paper identifies ELU as its best result at 72.86%. The closest paper comparison for the submitted best model is Swish at 70.63%, giving a difference of only 0.64 percentage points.

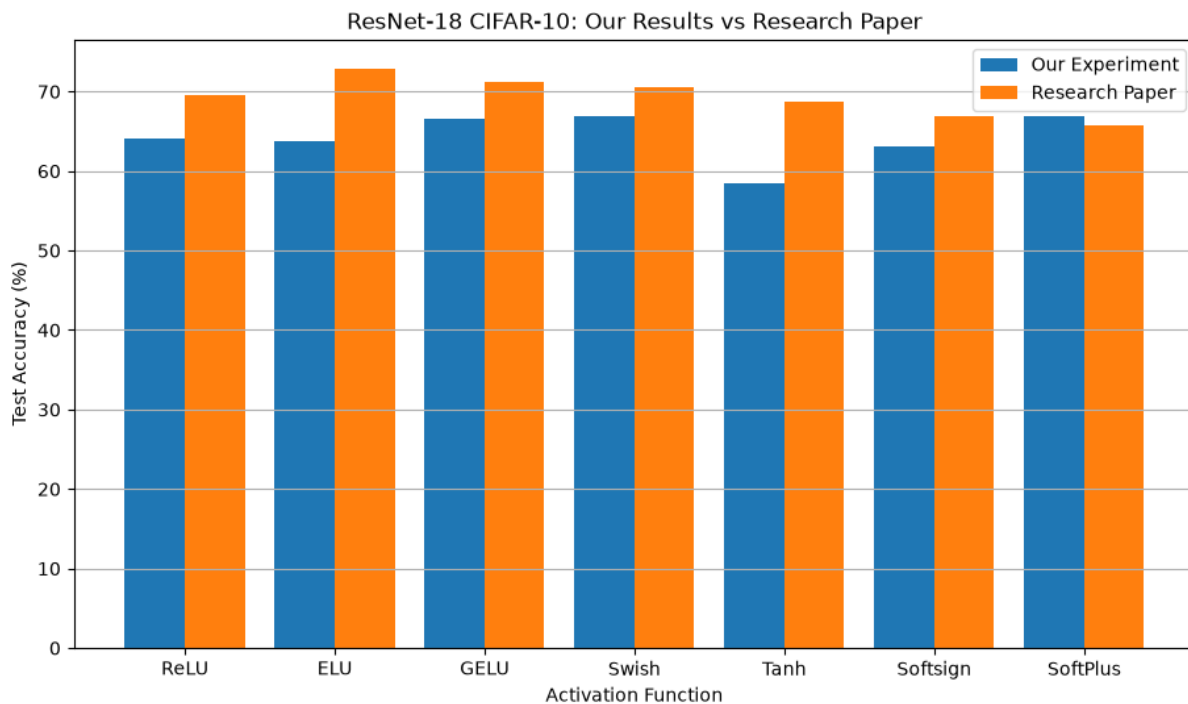


Figure 2. Accuracy comparison between the submitted experiment and the research paper.

11. Training-Time Comparison

Activation	Training time (minutes)
ReLU	9.41
Swish	10.00
Tanh	11.10
GELU	11.13
SoftPlus	12.15
ELU	16.35
Softsign	16.54

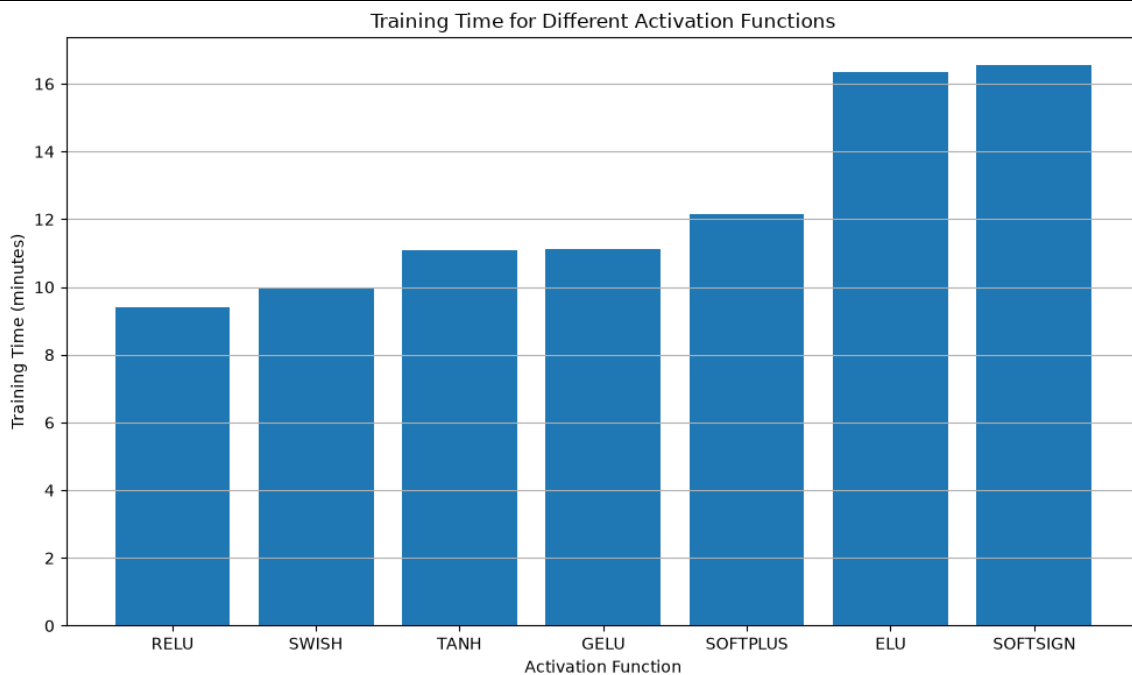


Figure 3. Training-time comparison generated by the notebook.

12. Selection of the Best Model

```
best_activation = max(  
    test_results,  
    key=test_results.get  
)
```

```
best_accuracy = test_results[best_activation]
```

OUTPUT

BEST ACTIVATION IN OUR EXPERIMENT

Activation: SWISH

Test Accuracy: 69.99%

Paper Accuracy: 70.63%

13. Detailed Evaluation of the Swish Model

The selected Swish model was evaluated on all 10,000 test images. The prediction vector has shape (10000,), matching the true-label vector. The classification report provides per-class precision, recall, F1-score, and support.

```
OUTPUT
Predictions generated successfully.
Prediction shape: (10000,)
True label shape: (10000,)
```

14. Classification Report

Class	Precision	Recall	F1-score	Support
airplane	0.7307	0.7650	0.7474	1000
automobile	0.8419	0.7240	0.7785	1000
bird	0.6059	0.6520	0.6281	1000
cat	0.5122	0.5050	0.5086	1000
deer	0.6649	0.6330	0.6486	1000
dog	0.6220	0.6040	0.6129	1000
frog	0.7278	0.7780	0.7521	1000
horse	0.8446	0.6900	0.7595	1000
ship	0.7554	0.8770	0.8117	1000
truck	0.7267	0.7710	0.7482	1000
Overall accuracy	0.6999	0.6999	0.6995	10000
Macro average	0.7032	0.6999	0.6995	10000
Weighted average	0.7032	0.6999	0.6995	10000

15. Confusion Matrix

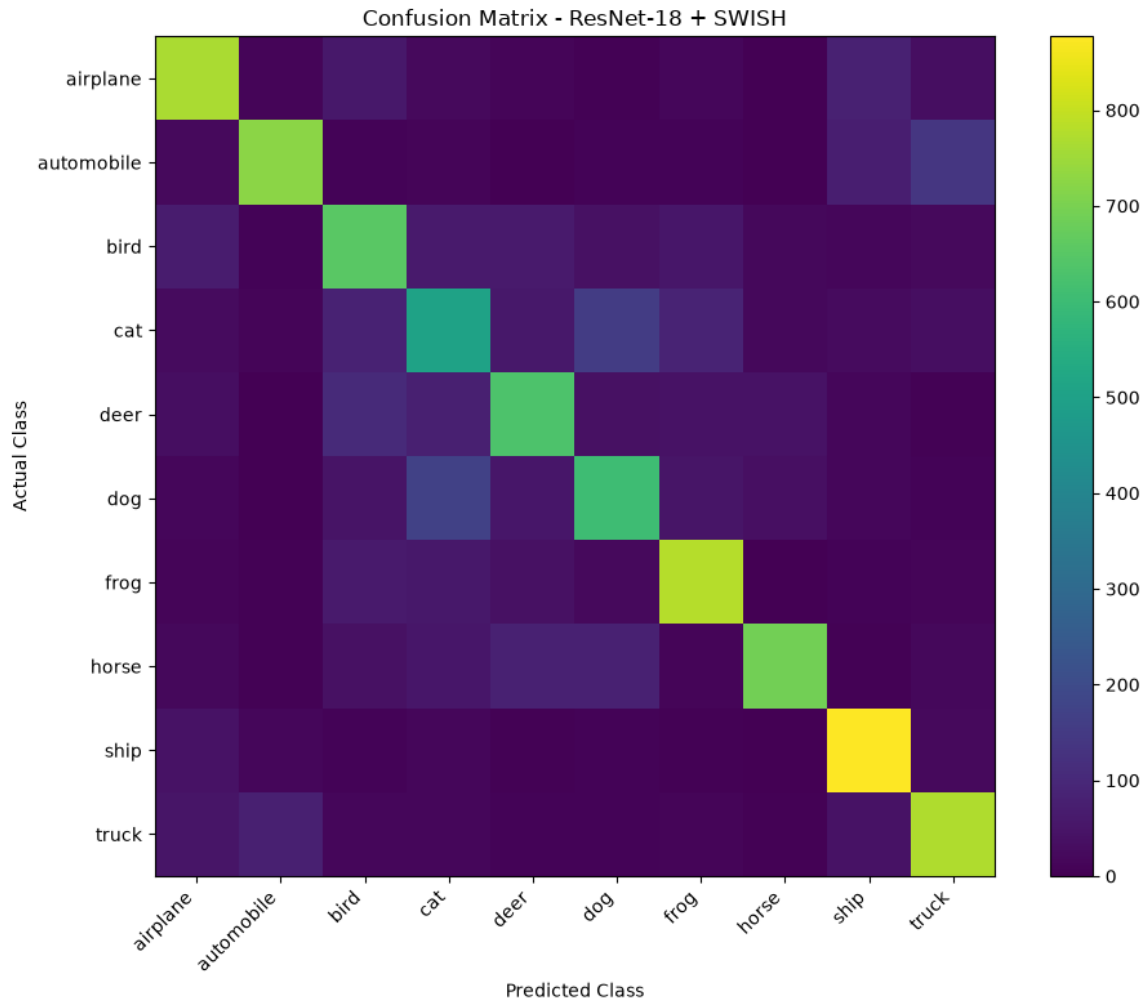


Figure 4. Confusion matrix for the final ResNet-18 + Swish model. Diagonal cells represent correct classifications; off-diagonal cells represent class confusions.

16. Sample Predictions

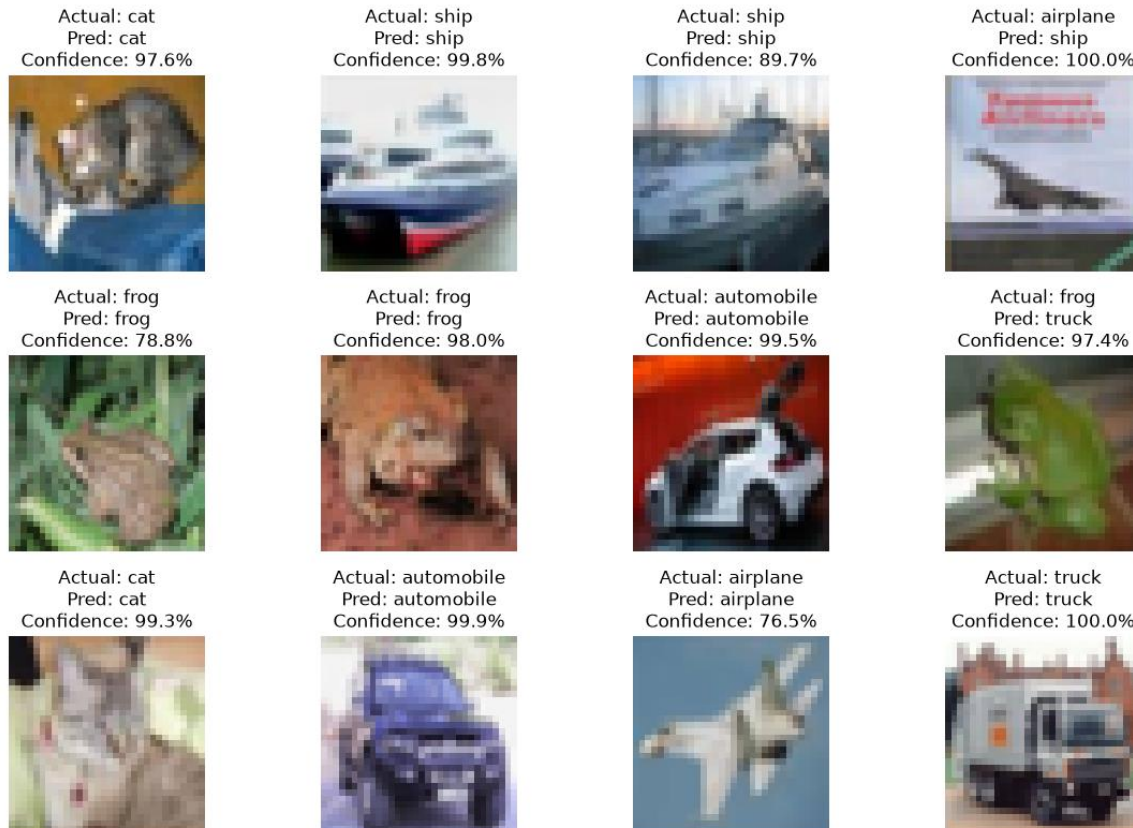


Figure 5. Sample test predictions with actual class, predicted class, and confidence.

17. Correct and Incorrect Predictions

```
OUTPUT
PREDICTION SUMMARY
Total test images: 10000
Correct predictions: 6999
Incorrect predictions: 3001
Accuracy: 69.99%
```

The final model correctly classified 6,999 of the 10,000 test images and misclassified 3,001 images.

18. Final Model

```
model_path = "ResNet18_CIFAR10_Swish.keras"
best_model.save(model_path)

OUTPUT
MODEL SAVED
Model: ResNet18_CIFAR10_Swish.keras
Activation: SWISH
Test Accuracy: 69.99%
```

19. Discussion

The final experiment demonstrates that activation choice changes both predictive performance and training time. Swish was the strongest activation in the submitted 15-epoch run, narrowly ahead of

ELU. The Swish result was close to the paper's reported Swish accuracy, while the paper's ELU result remained higher than the submitted ELU result.

SoftPlus is notable because its submitted accuracy (66.36%) was 0.57 percentage points higher than the paper's reported SoftPlus accuracy (65.79%). The largest negative difference was observed for Tanh, at -4.98 percentage points.

Training time also varied substantially. ReLU was the fastest among the seven final runs at approximately 9.41 minutes, while Softsign was the slowest at approximately 16.54 minutes. Swish required approximately 10.00 minutes and therefore provided the best accuracy in the experiment without being the most expensive activation in training time.

20. Important Experimental Scope and Limitations

The supplied paper describes the original CIFAR-10 split as 50,000 training images and 10,000 test images. The submitted notebook further divides the 50,000 training images into 45,000 training and 5,000 validation images. Therefore, the submitted experiment is not an exact reproduction of the paper's train/validation procedure.

The notebook's final comparative run uses 15 epochs. The supplied paper specifies the optimizer settings and reports the final activation-function accuracies, but the provided text does not establish a directly matching 15-epoch training configuration. Consequently, the results should be described as a controlled reproduction/implementation inspired by the paper rather than an exact numerical replication.

The notebook also contains earlier exploratory experiments with different epoch counts. The results in this report use the final 15-epoch comparative experiment for the main conclusion.

21. Conclusion

A ResNet-18 image-classification system was implemented and evaluated on CIFAR-10 with seven activation functions. Under the final 15-epoch configuration, Swish achieved the highest test accuracy of 69.99%. This is 0.64 percentage points below the paper's reported Swish accuracy of 70.63%. The final Swish model was evaluated with a full classification report, confusion matrix, sample predictions, and correct/incorrect prediction counts, and was saved as ResNet18_CIFAR10_Swish.keras.

For the academic presentation, the most important result is the comparison between the seven activation functions, followed by the detailed evaluation of the selected Swish model.

22. Key Code Used in the Final Experiment

```
# Final training configuration
EPOCHS = 15
BATCH_SIZE = 128
LEARNING_RATE = 0.1
MOMENTUM = 0.9
DECAY = 5e-4

# Fresh model for each activation
model = build_resnet18(
    input_shape=(32, 32, 3),
    num_classes=10,
    activation=activation_name
)

# Optimizer
lr_schedule = tf.keras.optimizers.schedules.InverseTimeDecay(
    initial_learning_rate=LEARNING_RATE,
    decay_steps=1,
    decay_rate=DECAY,
    staircase=False
)

optimizer = tf.keras.optimizers.SGD(
    learning_rate=lr_schedule,
    momentum=MOMENTUM
)

# Compile
model.compile(
    optimizer=optimizer,
    loss=tf.keras.losses.CategoricalCrossentropy(),
    metrics=["accuracy"]
)

# Train
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    verbose=1
)

# Evaluate
loss, accuracy = model.evaluate(
    x_test, y_test, verbose=1
)
```

23. References

Pandey, G. K., & Srivastava, S. “ResNet-18 comparative analysis of various activation functions for image classification.” Proceedings of the International Conference on Inventive Computation Technologies (ICICT 2023), IEEE, pp. 595–601, 2023.

The report's research-paper accuracy values, methodology descriptions, activation-function list, CIFAR-10 description, and optimizer settings are taken from the supplied paper.

Appendix A — Main Final Outputs

```
OUTPUT
===== FINAL COMPARISON =====
ReLU      : 66.41%
ELU       : 69.89%
GELU      : 68.73%
Swish     : 69.99%
Tanh      : 63.74%
Softsign  : 65.41%
SoftPlus  : 66.36%

===== BEST MODEL =====
Activation: SWISH
Test Accuracy: 69.99%
Paper Accuracy: 70.63%

===== FINAL PREDICTION SUMMARY =====
Total test images: 10000
Correct predictions: 6999
Incorrect predictions: 3001
Accuracy: 69.99%
```